

Real-world (score an answer that isn't exact) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
def build_judge_prompt(task, answer, rules):
    rules_text = "\n".join("- " + r for r in rules)
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text + "\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining"
    )

def score_output(task, answer, rules, call_model):
    prompt = build_judge_prompt(task, answer, rules)
    response = call_model(prompt)
    lines = response.strip().split("\n")
    verdict = lines[0].strip().upper()
    reason = " ".join(lines[1:]).strip()
    return {"verdict": verdict, "reason": reason}

# test
rules = ["mentions the correct category", "under 100 words"]
result = score_output("Classify this ticket", real_answer, rules, call_model)
assert result["verdict"] == "PASS", result["reason"]
```

This works, but note `call_model` is passed in directly — the test makes a real API call every time it runs, which costs money and can be slow.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a dataclass result, judge prompt kept as a separate versioned string

```
from dataclasses import dataclass

@dataclass
class JudgeResult:
    verdict: str
    reason: str

    @property
    def passed(self) -> bool:
        return self.verdict == "PASS"

JUDGE_PROMPT_VERSION = "v1"

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = "\n".join(f"- {r}" for r in rules)
    return (
        f"Task: {task}\n"
        f"Answer to check: {answer}\n"
        f"Rules the answer must follow:\n{rules_text}\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining"
    )

def score_output(task: str, answer: str, rules: list[str], call_model) -> JudgeResult:
    prompt = build_judge_prompt(task, answer, rules)
    response = call_model(prompt)
    lines = response.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    reason = lines[1].strip() if len(lines) > 1 else ""
    return JudgeResult(verdict=verdict, reason=reason)

def test_real_task_passes_judge():
    rules = ["mentions the correct category", "is under 100 words"]
```

```

answer = run_project_4(real_task)
result = score_output(real_task, answer, rules, call_model)
assert result.passed, result.reason

```

Approach 2 — a numeric score instead of PASS/FAIL, with a chosen bar

```

from dataclasses import dataclass

@dataclass
class JudgeResult:
    score: int # 1 to 5
    reason: str

PASS_BAR = 4

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = "\n".join(f"- {r}" for r in rules)
    return (
        f"Task: {task}\n"
        f"Answer to check: {answer}\n"
        f"Rules the answer should follow:\n{rules_text}\n"
        "Score the answer from 1 (fails badly) to 5 (fully meets every rule).\n"
        "Reply with the number on the first line, then one sentence explaining wh
    )

def score_output(task: str, answer: str, rules: list[str], call_model) -> JudgeRe
    prompt = build_judge_prompt(task, answer, rules)
    response = call_model(prompt)
    lines = response.strip().split("\n", 1)
    score = int(lines[0].strip())
    reason = lines[1].strip() if len(lines) > 1 else ""
    return JudgeResult(score=score, reason=reason)

def test_real_task_scores_above_bar():
    rules = ["mentions the correct category", "is under 100 words"]
    answer = run_project_4(real_task)
    result = score_output(real_task, answer, rules, call_model)
    assert result.score >= PASS_BAR, f"scored {result.score}: {result.reason}"

```

Difference from Basic: both Intermediate approaches wrap the raw dict in a typed dataclass, and split the judge prompt into its own versioned function so the exact wording sent to the judge is tracked, not rebuilt ad hoc. Approach 1's PASS/FAIL matches a normal test's mental model directly. Approach 2's 1-5 score gives you more information: a "3" and a "1" are both failures against a bar of 4, but a 3 tells you the answer was close, while a 1 tells you something's badly broken — useful when you're tracking quality trends over time, not just pass/fail on one run. Neither approach yet checks whether the judge itself can be trusted — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a calibration set: known-good and known-bad answers the judge must score correctly

```
from dataclasses import dataclass

@dataclass
class JudgeResult:
    verdict: str
    reason: str

    @property
    def passed(self) -> bool:
        return self.verdict == "PASS"

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = "\n".join(f"- {r}" for r in rules)
    return (
        f"Task: {task}\n"
        f"Answer to check: {answer}\n"
        f"Rules the answer must follow:\n{rules_text}\n"
        "Reply with PASS or FAIL on the first line, then one sentence explaining
    )

def score_output(task: str, answer: str, rules: list[str], call_model) -> JudgeResult:
    # temperature=0 (or as close as the model allows): a scoring call should be
    # as consistent as possible, not creative
    prompt = build_judge_prompt(task, answer, rules)
    response = call_model(prompt, temperature=0)
    lines = response.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    reason = lines[1].strip() if len(lines) > 1 else ""
    return JudgeResult(verdict=verdict, reason=reason)

TICKET_RULES = ["mentions the delivery issue", "is under 50 words", "does not inv
```

```

CALIBRATION_SET = [
    {
        "task": "Summarize this ticket about a late delivery.",
        "answer": "The customer's order shipped late due to a warehouse delay; th",
        "expected_verdict": "PASS",
    },
    {
        "task": "Summarize this ticket about a late delivery.",
        "answer": "I like pizza.",
        "expected_verdict": "FAIL", # obviously irrelevant
    },
    {
        "task": "Summarize this ticket about a late delivery.",
        "answer": "The customer's order arrived exactly on time and they were ext",
        "free upgrade to overnight shipping that our team proactively arranged fo",
        "expected_verdict": "FAIL", # confidently states facts that contradict t
    },
]

def test_judge_calibration() -> None:
    for case in CALIBRATION_SET:
        result = score_output(case["task"], case["answer"], TICKET_RULES, call_mo
        assert result.verdict == case["expected_verdict"], (
            f"judge got a known-{case['expected_verdict']} case wrong: {result.re
        )

```

Expected output: all 3 calibration cases pass. If the second or third case fails — the judge says `PASS` on the pizza answer or the fabricated on-time story — that's not a minor issue, it means the judge cannot currently be trusted to gate anything, and the judge prompt itself (not your real Project 4 tasks) is what needs fixing first.

Approach 2 — self-consistency check

```

def test_judge_is_self_consistent() -> None:
    task = "Summarize this ticket about a late delivery."
    answer = "The customer's order shipped late due to a warehouse delay; they we

    verdicts = [
        score_output(task, answer, TICKET_RULES, call_model).verdict
        for _ in range(3)
    ]

    assert len(set(verdicts)) == 1, f"judge disagreed with itself across identical

```

Expected output: `verdicts == ["PASS", "PASS", "PASS"]`, so `set(verdicts)` has exactly 1 item. If the judge flips between calls on the exact same input, `temperature=0` isn't fully eliminating randomness for this model, or the judge prompt is ambiguous enough that borderline reasoning goes either way — either way, that's noise you don't want feeding into a real regression check.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate builds a judge that works, and trusts it. Approach 1 doesn't touch the judge's own code at all — it adds a test of the judge, using cases where the right answer is obvious to a human, so a judge that fails calibration is caught before it ever scores a real, ambiguous Project 4 output. Approach 2 checks a different property: that the judge doesn't contradict itself on identical input, which calibration alone wouldn't catch (a judge could pass every calibration case on one lucky run while still being noisy in general).

Which one should you actually write? Both, and run them once whenever you change the judge prompt — not on every single eval run, since they cost extra API calls for no new information once the judge prompt hasn't changed. Approach 1's calibration set is the more important of the two: it directly answers "can I trust this judge at all," and a 4-6 case set costs almost nothing to build once, using answers you already know are obviously right or wrong. Approach 2's self-consistency check is worth adding once you've actually seen a judge flip-flop on a rerun — it's cheap insurance against a specific, known LLM-as-judge failure mode, not something every project needs from day one.